

Mise en place d'Exegol

1. Installation de Docker sur Debian

Pour utiliser Exegol, je commence par installer Docker sur une VM Debian en suivant la méthode officielle pour Debian.

On commence par mettre à jour et installer les prérequis avec :

```
sudo apt update
sudo apt install -y ca-certificates curl gnupg
sudo install -m 0755 -d /etc/apt/keyrings
```

Puis on ajoute la clé GPG Docker et du dépôt officiel :

```
curl -fsSL https://download.docker.com/linux/debian/gpg | sudo gpg --dearmor -o
/etc/apt/keyrings/docker.gpg
```

```
echo \
"deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg]
https://download.docker.com/linux/debian \
$(. /etc/os-release && echo $VERSION_CODENAME) stable" | \
sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

Enfin on installe les paquets Docker :

```
sudo apt update
sudo apt install -y docker-ce docker-ce-cli containerd.io docker-buildx-plugin
docker-compose-plugin
```

Pour vérifier si docker est bien installé et fonctionne, on utilise ces commandes :

```
docker --version
docker compose version
sudo systemctl status docker --no-pager
```

```
root@debian13vm:~# docker --version
Docker version 29.1.5, build 0e6fee6
root@debian13vm:~# docker compose version
Docker Compose version v5.0.1
root@debian13vm:~# sudo systemctl status docker --no-pager
● docker.service - Docker Application Container Engine
   Loaded: loaded (/usr/lib/systemd/system/docker.service; enabled; preset: enabled)
   Active: active (running) since Mon 2026-01-19 15:52:20 CET; 7min ago
     Invocation: b0bd97d3e02c468e9eef64e71b6fb7d6
  TriggeredBy: ● docker.socket
       Docs: https://docs.docker.com
    Main PID: 2592 (dockerd)
       Tasks: 7
      Memory: 24.6M (peak: 26.9M)
         CPU: 399ms
    CGroup: /system.slice/docker.service
            └─2592 /usr/bin/dockerd -H fd:// --containerd=/run/containerd/containerd.sock

janv. 19 15:52:19 debian13vm dockerd[2592]: time="2026-01-19T15:52:19.478086620+01:00" level=info msg="Restoring contain... start."
janv. 19 15:52:19 debian13vm dockerd[2592]: time="2026-01-19T15:52:19.491923405+01:00" level=info msg="Deleting nftable...tatus 1"
janv. 19 15:52:19 debian13vm dockerd[2592]: time="2026-01-19T15:52:19.505679815+01:00" level=info msg="Deleting nftable...tatus 1"
janv. 19 15:52:20 debian13vm dockerd[2592]: time="2026-01-19T15:52:20.099023008+01:00" level=info msg="Loading containe... done."
janv. 19 15:52:20 debian13vm dockerd[2592]: time="2026-01-19T15:52:20.114961166+01:00" level=info msg="Docker daemon" c.n=29.1.5
janv. 19 15:52:20 debian13vm dockerd[2592]: time="2026-01-19T15:52:20.115240937+01:00" level=info msg="Initializing buildkit"
janv. 19 15:52:20 debian13vm dockerd[2592]: time="2026-01-19T15:52:20.135323765+01:00" level=info msg="Completed buildk...ization"
janv. 19 15:52:20 debian13vm dockerd[2592]: time="2026-01-19T15:52:20.140912254+01:00" level=info msg="Daemon has compl...ization"
```

Pour éviter d'utiliser systématiquement **sudo** avec Docker, on ajoute l'utilisateur courant au groupe docker avec :

```
sudo usermod -aG docker $USER  
newgrp docker
```

Afin de vérifier le bon fonctionnement de Docker avec un conteneur de test, on utilise la commande : **docker run --rm hello-world**

```
root@debian13vm:~# docker run --rm hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17eec7bbc9d7: Pull complete
Digest: sha256:05813aedc15fb7b4d732e1be879d3252c1c9c25d885824f6295cab4538cb85cd
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
 1. The Docker client contacted the Docker daemon.
 2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
    (amd64)
 3. The Docker daemon created a new container from that image which runs the
    executable that produces the output you are currently reading.
 4. The Docker daemon streamed that output to the Docker client, which sent it
    to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/
```

→ Cette commande affiche le message "Hello from Docker!", ce qui confirme que le démon docker fonctionne correctement.

2. Installation d'Exegol avec pipx

Pour installer Exegol proprement, j'utilise **pipx** et un environnement Python dédié.

On commence par installer les paquets nécessaires avec **sudo apt install -y pipx python3-venv**

Puis on ajoute le binaire pipx au PATH avec **pipx ensurepath**

```

student@debian13vm: $ sudo apt install -y pipx python3-venv
pipx est déjà la version la plus récente (1.7.1-1).
python3-venv est déjà la version la plus récente (3.13.5-1).
Sommaire :
  Mise à niveau de : 0. Installation de : 0Supprimé : 0. Non mis à jour : 149
student@debian13vm: $ pipx ensurepath
Success! Added /home/student/.local/bin to the PATH environment variable.

Consider adding shell completions for pipx. Run 'pipx completions' for instructions.

You will need to open a new terminal or re-login for the PATH changes to take effect. Alternatively, you can source your
shell's config file with e.g. 'source ~/.bashrc'.

Otherwise pipx is ready to go! ✨ 🌟 ✨

```

Je travaille ensuite sous le compte **student** qui est utilisé pour l'exploitation, tandis que les privilèges root sont réservés à l'administration, conformément au principe du moindre privilège.

Je vérifie que le chemin **~/local/bin** est bien présent dans la variable PATH avec :

```

student@debian13vm:~$ echo "$PATH" | tr ':' '\n' | grep -n "\.local/bin" || echo "PAS TROUVÉ"
7:/home/student/.local/bin

```

L'installation d'Exegol se fait ensuite via pipx : **pipx install exegol**

```

student@debian13vm:~$ pipx install exegol
  installed package exegol 5.1.8, installed using Python 3.13.5
  These apps are now globally available
    - exegol
done! ✨ 🌟 ✨

```

On vérifie l'installation avec :

pipx list

```

student@debian13vm:~$ pipx list
venvs are in /home/student/.local/share/pipx/venvs
apps are exposed on your $PATH at /home/student/.local/bin
manual pages are exposed at /home/student/.local/share/man
  package exegol 5.1.8, installed using Python 3.13.5
    - exegol

```

exegol version

```

student@debian13vm: $ exegol version
[*] Exegol Community is currently in version v5.1.8
[?] I confirm that I've read and accepted the EULA (https://docs.exegol.com/legal/eula) [y/N]: y
[?] Do you want to activate your Exegol subscription now? [y/N]: N
[!] Community plan (i.e., free) is strictly limited to personal, non-commercial, educational, or research purposes.
[?] I confirm that I'm not trying to rip off the developers, who put their blood, sweat, and tears into Exegol, and that I would
absolutely support the project if I were using it professionally. I'm not *that* kind of person. [y/N]: y
[+] You can now use Exegol Community Edition! Enjoy :)
[*] Exegol Community (personal use only)
[*] More about Exegol at: https://exegol.com
student@debian13vm: $

```

3. Préparation des workspaces et téléchargement de l'image Exegol

On va préparer une structure de travail pour séparer les données du lab GOAD et les notes :

```
mkdir -p ~/exegol/workspaces/goad
```

```
mkdir -p ~/exegol/notes
```

On vérifie la création des répertoires avec : **ls -la ~/exegol**

```
student@debian13vm:~$ mkdir -p ~/exegol/workspaces/goad
student@debian13vm:~$ mkdir -p ~/exegol/notes
student@debian13vm:~$ ls -la ~/exegol
total 16
drwxrwxr-x 4 student student 4096 19 janv. 16:59 .
drwx----- 7 student student 4096 19 janv. 16:59 ..
drwxrwxr-x 2 student student 4096 19 janv. 16:59 notes
drwxrwxr-x 3 student student 4096 19 janv. 16:59 workspaces
```

Pour télécharger une image Exegol, on utilise la commande : **exegol install**

Lors du premier téléchargement, une erreur “no space left on device” apparaît, ce qui révèle que, même si le disque virtuel a été étendu à 120 Go, la partition système /dev/sda3 est encore limitée à 60 Go.

Après diagnostic avec **df -h** et **lsblk**, j'étends la partition à l'aide de **growpart /dev/sda 3**, puis j'agrandis le système de fichiers avec **resize2fs /dev/sda3**.

```
student@debian13vm:~$ sudo growpart /dev/sda 3
CHANGED: partition=3 start=17164288 old: size=126193664 end=143357951 new: size=234493919 end=251658206
student@debian13vm:~$ sudo resize2fs /dev/sda3
resize2fs 1.47.2 (1-Jan-2025)
Filesystem at /dev/sda3 is mounted on /; on-line resizing required
old_desc_blocks = 8, new_desc_blocks = 14
The filesystem on /dev/sda3 is now 29311739 (4k) blocks long.

student@debian13vm:~$ df -h /
Sys. de fichiers Taille Utilisé Dispo Uti% Monté sur
/dev/sda3      110G   18G   88G  17% /
student@debian13vm:~$ lsblk
NAME MAJ:MIN RM SIZE RO TYPE MOUNTPOINTS
sda   8:0    0  120G  0 disk
├─sda1 8:1    0  190M  0 part /boot
├─sda2 8:2    0    8G  0 part [SWAP]
└─sda3 8:3    0 111,8G  0 part /
```

En lançant l'installation de Exegol, le menu affiche différentes images disponibles (free, web, osint, light, ad, full, nightly) avec leur taille et leur statut, la plupart étant réservées aux versions Pro/Enterprise.

On sélectionne l'image **free**, qui pèse environ 42,9 Go une fois installée..

```
student@debian13vm:~$ exegol install
[*] Exegol Community is currently in version v5.1.8
[*] More about Exegol at: https://exegol.com

🐳 Available images
```

Image	Size	Status
free	42.91 GB	Not installed
web	18.94 GB	Pro / Enterprise only
osint	11.3 GB	Pro / Enterprise only
light	14.72 GB	Pro / Enterprise only
ad	28.67 GB	Pro / Enterprise only
full	42.59 GB	Pro / Enterprise only
nightly	42.5 GB	Pro / Enterprise only

```

[?] Select an image by its name (free): free
[*] Installing exegol image : free
[*] Pulling compressed image, starting download 🐳
[*] Once downloaded and uncompressed, the image will take ~42.91 GB on disk 📦
[+] Status: Downloaded newer image for nwodtuhs/exegol:free
[+] Image successfully installed

```

On vérifie enfin le statut d'Exegol avec : **exegol info** et **docker images | grep -i exegol** .

```
student@debian13vm:~$ exegol info
[*] Exegol Community is currently in version v5.1.8
[*] More about Exegol at: https://exegol.com

🐳 Available images
```

Image	Size	Status
free	42.9 GB	Up to date (v.3.1.8)
web	18.94 GB	Pro / Enterprise only
osint	11.3 GB	Pro / Enterprise only
light	14.72 GB	Pro / Enterprise only
ad	28.67 GB	Pro / Enterprise only
full	42.59 GB	Pro / Enterprise only
nightly	42.5 GB	Pro / Enterprise only

```

student@debian13vm:~$ docker images | grep -i exegol
WARNING: This output is designed for human readability. For machine-readable output, please use --format.
nwodtuhs/exegol:free        67071a520a08         46.1GB              0B

```

On voit que l'image free est Up to date et disponible localement dans Docker.

Dans le conteneur Exegol, on configure un workspace dédié à GOAD en utilisant le répertoire **~/exegol/workspaces/goad** comme point de montage.

On crée par exemple un fichier **preuvieworkspace.txt** depuis le conteneur, puis on vérifie sa présence sur l'hôte dans **~/exegol/workspaces/goad**, ce qui prouve la persistance des données entre les sessions Exegol.

```

[Jan 20, 2026 - 09:11:47 (CET)] exegol-goad-red bloodhound # cd /workspace
[Jan 20, 2026 - 09:48:09 (CET)] exegol-goad-red /workspace # echo "preuve workspace $(date)" > preuve_workspace.txt
[Jan 20, 2026 - 09:48:14 (CET)] exegol-goad-red /workspace # ls -la
total 14428
drwxrwxr-x 3 1001 1001      4096 Jan 20 09:48 .
drwxr-xr-x 1 root root      4096 Jan 20 09:02 ..
drwxrwxr-x 2 1001 1001      4096 Jan 20 09:11 bloodhound
-rw----- 1 root root 105934848 Jan 20 09:06 core.2166
-rw-rw---- 1 root root      49 Jan 20 09:48 preuve_workspace.txt
[Jan 20, 2026 - 09:48:19 (CET)] exegol-goad-red /workspace # exit
student@debian13vm: $ ls -la ~/exegol/workspaces/goad
total 14428
drwxrwxr-x 3 student student      4096 20 janv. 09:48 .
drwxrwxr-x 3 student student      4096 19 janv. 16:59 ..
drwxrwxr-x 2 student student      4096 20 janv. 09:11 bloodhound
-rw----- 1 root root 105934848 20 janv. 09:06 core.2166
-rw-rw---- 1 root root      49 20 janv. 09:48 preuve_workspace.txt
student@debian13vm: $ cat ~/exegol/workspaces/goad/preuve_workspace.txt
cat: /home/student/exegol/workspaces/goad/preuve_workspace.txt: Permission non accordée
student@debian13vm: $ sudo cat ~/exegol/workspaces/goad/preuve_workspace.txt
preuve workspace Tue Jan 20 09:48:14 AM CET 2026

```

On vérifie ensuite la présence et le fonctionnement de plusieurs outils orientés Red Team intégrés dans Exegol, tels que :

- **nmap --version** pour le scanning réseau,
- **python3 --version** pour les scripts,
- **netexec --help** pour l'audit d'Active Directory.

Exegol fournit ainsi un environnement complet contenant de nombreux outils utiles pour l'audit d'AD, qui seront utilisés plus tard sur le lab GOAD.

```

[Jan 20, 2026 - 09:58:13 (CET)] exegol-goad-red /workspace # nmap --version
Nmap version 7.93 ( https://nmap.org )
Platform: x86_64-pc-linux-gnu
Compiled with: liblua-5.3.6 openssl-3.0.16 libssh2-1.10.0 libz-1.2.13 libpcap-1.10.3 nmap-libdnet-1.12 ipv6
Compiled without:
Available nsock engines: epoll poll select
[Jan 20, 2026 - 09:58:15 (CET)] exegol-goad-red /workspace # python3 --version
Python 3.11.13
[Jan 20, 2026 - 09:58:19 (CET)] exegol-goad-red /workspace # impacket-smbclient -h | head
zsh: command not found: impacket-smbclient
[Jan 20, 2026 - 09:58:23 (CET)] exegol-goad-red /workspace # crackmapexec --help | head
crackmapexec has been replaced with netexec --help
[Jan 20, 2026 - 09:58:28 (CET)] exegol-goad-red /workspace # netexec --help | head
usage: netexec [-h] [--version] [-t THREADS] [--timeout TIMEOUT]
               [--jitter INTERVAL] [--verbose] [--debug] [--no-progress]
               [--log LOG] [-6] [--dns-server DNS_SERVER] [--dns-tcp]
               [--dns-timeout DNS_TIMEOUT]
               {wmi,ftp,smb,ldap,rdp,ssh,vnc,nfs,mssql,winrm} ...

      .| .|
      || ||
      \ \ )//

```